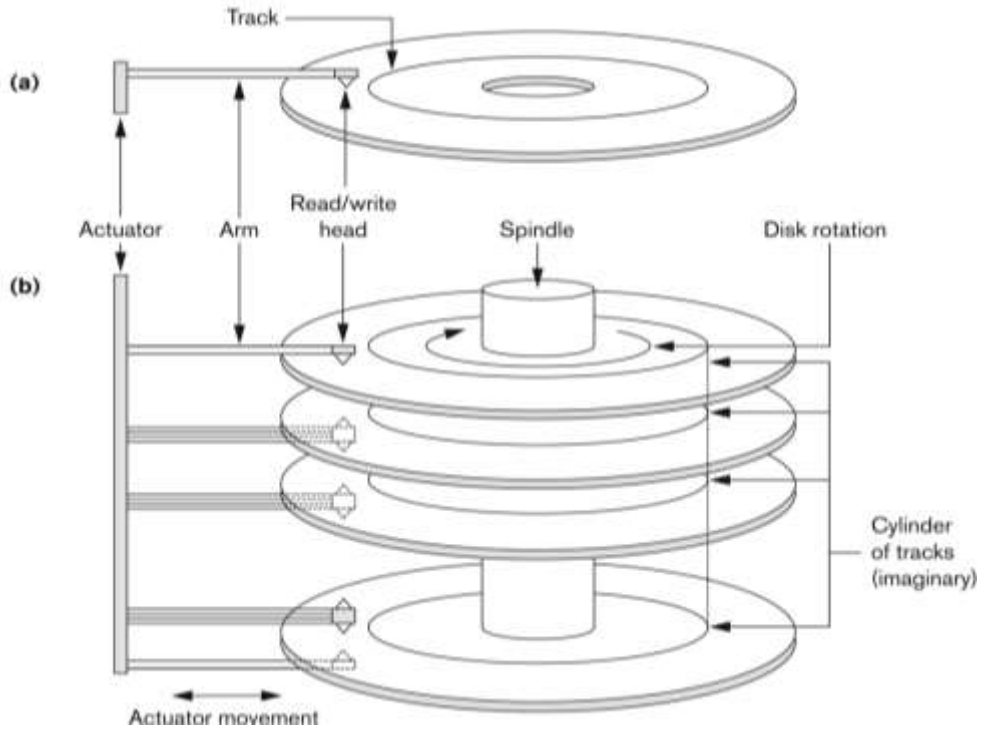
File Organization

- The computerized db must be stored physically on some computer storage medium. There are two main categories;
    1. **Primary Storage**

       This can be operated directly by the CPU. Provides fast access to data but is of limited storage capacity.
    2. **Secondary Storage**

       Includes magnetic disks, optical disks and tapes. Have a large capacity less cost and provide slower access to data than primary storage. Cannot be directly processed by the CPU. Must be copied into primary storage.
- **Hardware Description: (a) Single sided disk with read/write hardware. (b) disk pack with read/write hardware**



CPU ->prim->Sec

- The <u>division of track into equal sized disk blocks (or pages)</u> is set by the operating system during disk formatting or initialization.
- Blocks are separated by fixed size <u>interblock gap</u>s, which include specially coded control information written during disk initialization. This information is used to determine which block on the track follows each interblock gap.
- A disk is a random access addressable device.
- Transfer of data between main memory and disk takes place in units of disk blocks.

- The hardware address of a block is
    - Cylinder number, track number, block number (Within the track)
    - In many modern disk drives, a single number called LBA (Logical Block Address) is used.
    - A disk controller controls the disk drive and interfaces it to the computer system.

Parameters of the disks

Three time Components:

1. Seek Time

   To transfer a disk block given its address, the disk controller must first mechanically position the read/write head on the correct track. The time required for this is the seek time. Disk manufacturers provides an average seek time in milliseconds (10 to 60 msec)

2. Rotational Delay (Latency)

   Once when the read/write head is at the correct track, the time taken for the required block to rotate into position under the read/write head.

   If the speed of the disk rotation is *p* revolutions per minute (rpm) then the average rotational delay *rd*:

   $rd = (1/2)*(1/p)$ min = (60*1000)/(2*p) msec

3. Block Transfer Time (*btt*)

   The time needed to transfer the data in the block. This depends on the block size, track size and the rotational delay. If the transfer rate for the disk is *tr* bytes/msec and the block size is B bytes then:

   $btt = B/tr$ msec

   Track Size = 50 Kbytes *p* is 3600 rpm then the *tr* in bytes/msec is

   $tr = (50*1000)/(60*1000/3600) = 3000$bytes/msec

   $btt = B/3000$ msec

4. The average time needed to find and transfer a block, given its block address

   – $(s+rd+btt)$ msec

5. To transfer consecutively *k* non-contiguous blocks that are on the same cylinder

   – $(s+k*(rd+btt))$ msec

6. To transfer consecutively *k* contiguous blocks that are on the same track/cylinder

   – $(s+rd+(k*btt))$ msec

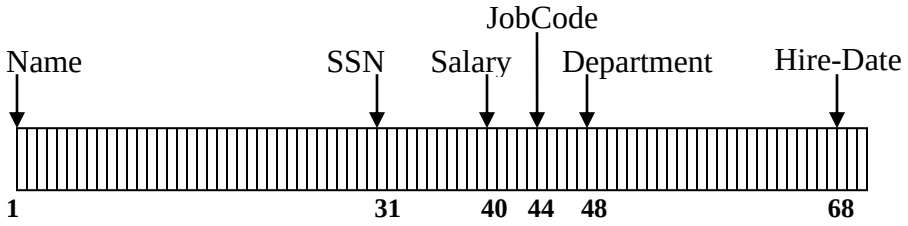
Placing file records on disk

- A file is a *sequence* of records, where each record is a collection of data values (or data items).
- A file descriptor (or file header) includes information that describes the file, such as the *field names* and their *data types*, and the addresses of the file blocks on disk.

- Records contain fields which have values of a particular type

  E.g., amount, date, time, age
- Records are stored on disk blocks.
- The blocking factor *bfr* for a file is the (average) number of file records stored in a disk block.
- A file can have fixed-length records (every record in the file has exactly the same size) or variable-length records.

A fixed length record with six fields of 71 bytes



Reasons for variable length records:

One or more fields are of varying size (don't know the exact length of the field). Eg: Name, One or more may be optional, file may have different record types.

Separator Characters: separates field name from field value. (normally use with variable length records)

Spanned Vs Unspanned

- Records of a file must be allocated to disk blocks.
- Block size B bytes, record size R bytes with B>=R
  - Blocking factor *bfr* = $\left\lfloor \frac{B}{R} \right\rfloor$ Records per block

    $\lfloor x \rfloor$ - floor function, rounds down the number x to an integer.
  - In general R may not devide B exactly, unused space = B - (*bfr**R) bytes

  Block = 10 byte

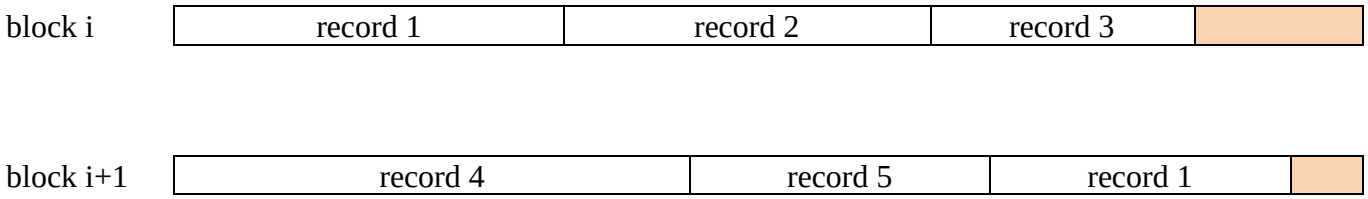
  R = 3 byte

  Bfr=B/R=3

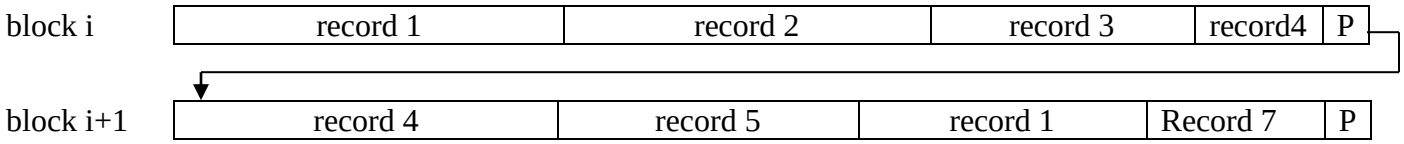
  1 byte
- To utilize unused space, store part of the record in one block and the rest in another.
- If consecutive blocks are not used, a pointer at the end of the first block points to the block which has the rest of records.

- This organization is called spanned.
- If records are not allowed to cross boundaries, the organization is called unspanned.
    - Used for fixed length records having B>R.

Unspanned Organization



Spanned Organization



- *bfr* is used to calculate the number of blocks (*b*) need to store a file.
- If the file has *r* records
    - $b = \left\lceil \frac{r}{bfr} \right\rceil$
    - $\lceil x \rceil$ ceiling function, rounds the value x up to the next integer.

File Header (file descriptor)

- Contains information about a file that is needed by the system programs that access the file records.
- Includes information:
    - To determine the disk address.
    - Record format description.

Heap Files (pile files, unordered files)

- Simplest and the most basic type of organization.
- Records are unordered.
- New records are inserted at the end of the file
- Can use spanned or unspanned organization.
- Inserting a record.
    - Very efficient, the last block of the file is copied into the buffer, the new record is added, and the block is written back to disk.
    - The address of the last block is kept in the file header.
- Searching a record
    - Involves a linear search, expensive procedure.

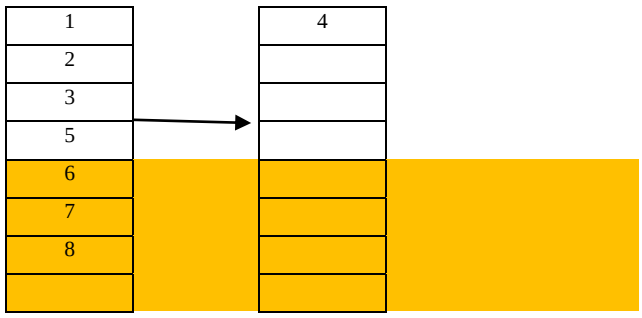
    - Requires searching (b/2) blocks on average.
    - If no records or several records satisfy the search then all b blocks must be searched.
- Deleting a record
    - Find the block, copy the block into the buffer, delete the record, rewrite the block to the disk.
    - Leave wasted storage space.
    - Expensive operation.

Techniques used in deleting

- Use the space when inserting new records.
    - This needs to keep track of empty locations.
- Have an extra bit/byte, called a **deletion marker** stored with each record.
    - When a record is deleted set a deletion marker to a certain value.
    - When searching consider only valid records.
    - Require periodic reorganization to reclaim unused space.
- For an unordered file, with fixed length records using unspanned blocks, and contiguous allocation can access any records by its position.
    - File records are numbered 0, 1, 2, …, r-1
    - Records in each block are numbered 0, 1, 2, …, *bfr*-1.
    - The ith record of the file is located in block $\left\lfloor \frac{i}{bfr} \right\rfloor$ and is the $(i \bmod bfr)^{th}$ record in that block.
    - Such files are known as relative or direct file.
    - Does not help to locate records on a search condition.

Sequential Files

- Records are ordered based on the values of the field – ordering field.
- Ordering field may be the key field.
- Reading the records in the order of the ordering key value becomes efficient.
- Finding the next record from the current one is efficient. The next record is in the same block unless it is the last record.
- Searching records in the ordered field is very efficient.
    - Can use binary search.
    - Usually access $\log_2(b)$ blocks.
- Does not provide any advantage for random or ordered access based on non-ordering field.
- Inserting records is expensive.
    - Find the correct position.
    - Must make space in the file to insert the record.



Techniques

- Keep unused space in each block.
    - Once the space is used, original problem resurfaces.
- Create a temporary unordered file.
    - This is known as **overflow or transaction file**.
    - Actual ordered file is known as main or master file.
    - New records are inserted at the end of the overflow file.
    - Periodically the overflow file is sorted and merged with the master file.
    - Insertion becomes efficient, but searching is complex.
        - Overflow file must be searched using linear search.
        - But for application that do not require up-to-date information, searching the overflow file could be ignored.

Hashing Techniques

- Primary file organization based on hashing.
- This organization is called a hash file.
- Provides fast access to records on certain search conditions.
- The search condition must be an equality condition on a single field. This field is called the hash field
- If the hash field is the key field then it is called the hash key.
- Provides a function h, called a hash function (randomizing function), that is applied to the hash field value of a record and yields the address of the block in which the record is stored.
- A search for the record is done in the main memory buffer.
- For most records, we need only a single block access.
- Hashing is also used as an internal search structure.

Internal Hashing

- Implemented as a hash table through the use of an array of records.

- Suppose that the array index ranges from 0 to M-1, then we have M slots whose address correspond to the array index.
- Choose a hash function that transforms the hash field value to an integer between 0 and M-1.
- One common hash function is:
    - H(K) = K mod M, K-hash field value

	Name	SSN	Job	Salary
0				
1				
2				
3				
		. .		
M-2				
M-1				

- Non integer hash field values can be transformed into integers before the hash function is applied.
- Use the ASCII values – multiply them.

Hash function to a character string K

```
Temp ← 1;
For i ← 1 to 20 do
        Temp ← Temp * code(K[i]) mod M;
Hash_address ← Temp mod M;
```

- The problem with hashing is, it may not guarantee distinct values.
- A **collision** occurs when the hash field value of a record that is been inserted hashes to an address that already contains a different record.
- In such case must insert the new record in some other position.
- The process of finding another position is called collision resolution.

Methods for collision resolution

- **Open addressing**: Proceeding from the occupied position specified by the hash address, the program checks the subsequent positions in order until an unused (empty) position is found.
- **Chaining**: For this method, various overflow locations are kept, usually by extending the array with a number of overflow positions. In addition, a pointer field is added to each record location. A collision is resolved by placing the new record in an unused overflow location and setting the pointer of the occupied hash address location to the address of that overflow location.

- **Multiple hashing**: The program applies a second hash function if the first results in a collision. If another collision results, the program uses open addressing or applies a third hash function and then uses open addressing if necessary.

	Data Fields	Overflow Pointer	
0		-1	Address Space
1		M	
2		-1	
3		-1	
4		M+2	
	.		
	.		
M-2		M+1	
M-1		-1	
M		M+5	Overflow Space
M+1		-1	
M+2		M+4	
	.		
	.		
M+0-2			
M+0-1			

H(Key)=Key mod 5

H(5) = 5 mod 5 =0

H(4) = 4 mod 5 =4

null pointer = -1

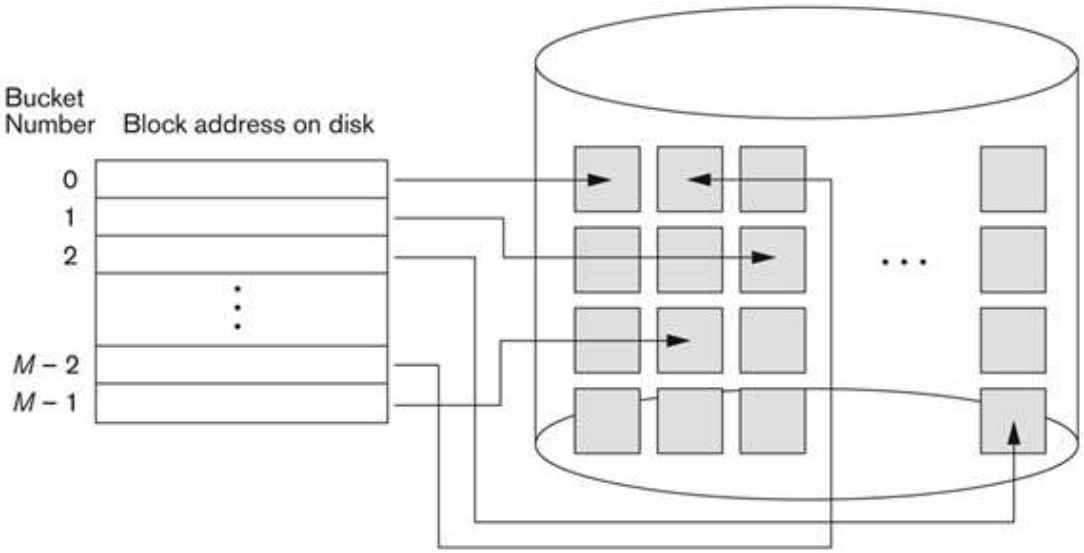
overflow pointer refers to position of next record in linked list.

External Hashing for disk files

- Hashing for disk files is called external hashing.

Static Hashing

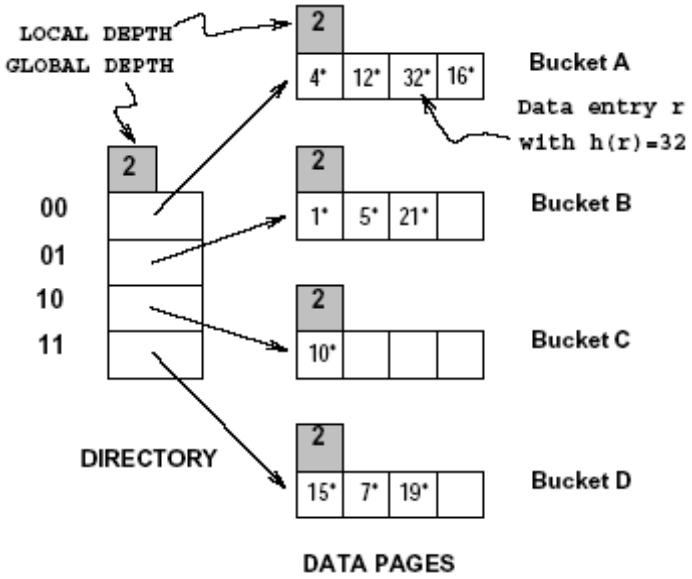
- Address space is made of buckets, holds multiple records.
- A bucket is either one disk block or a cluster of contiguous block.
- The hashing function maps a key into a relative bucket number.
- Using the file header the bucket number is converted into the corresponding disk block address.



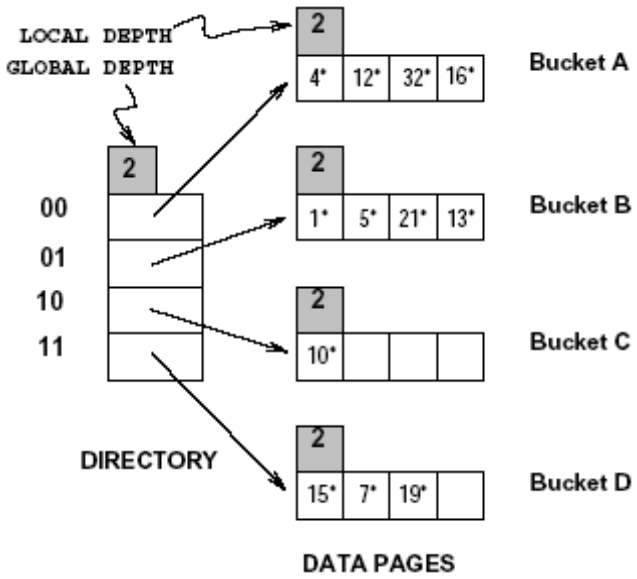
- Collision is less severe.
    - Many records will fit in a bucket.
- Collision occurs if the buckets are filled to its capacity.
- Use a variation of chaining where a pointer is maintained in neach bucket to a linked list of overflow records for the bucket.
- In this method a fixed number of M is allocated.

Dynamic Hashing & Extendible Hashing

- **Extendible Hashing**
    - Uses a type of directory - an array of 2d bucket address.
    - D is known as the global depth.
    - The integer value corresponding to the first d bits of the hash value is used as an index to the array to determine a directory entry, and the address in that entry determines the bucket.
- The directory consists of an array size 4 (22).
- Each element in the array is a pointer to the bucket.

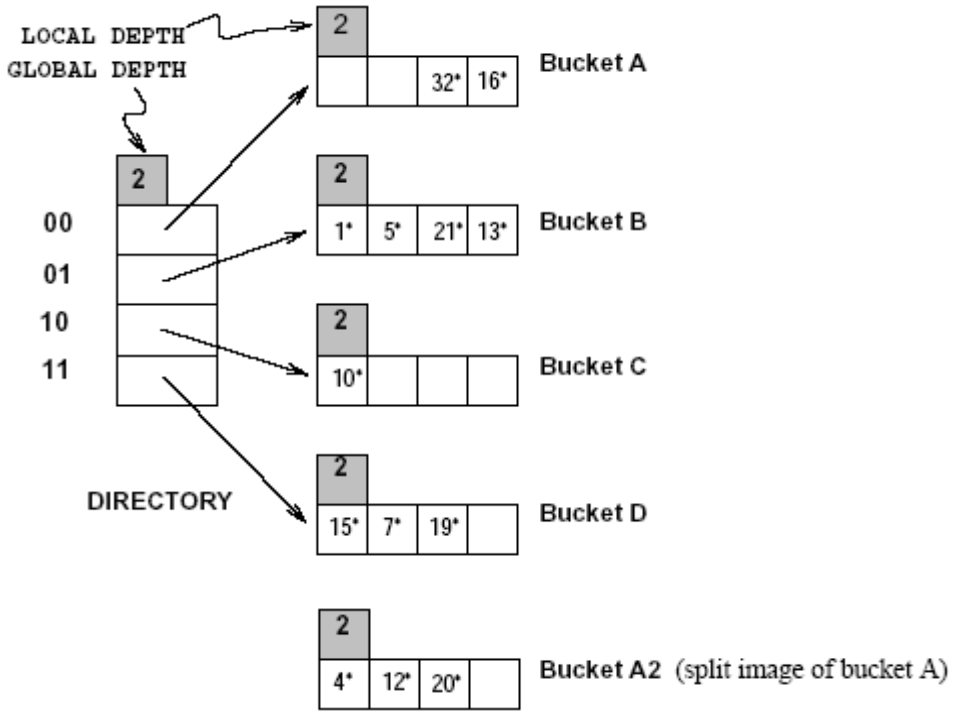


- To locate an entry (record) apply the hash function to the search field, and take the last two bits (because d is 2) of its binary representation.
- Locate the entry with hash value 5.
    - Binary code 101.
    - Look at directory entry 01 and follow the pointer.
    - Insert a data entry with hash value 13.
        - Take the binary code 1101.
        - Consider the last two bits 01.
        - Go to bucket B.
        - Page has space insert it.

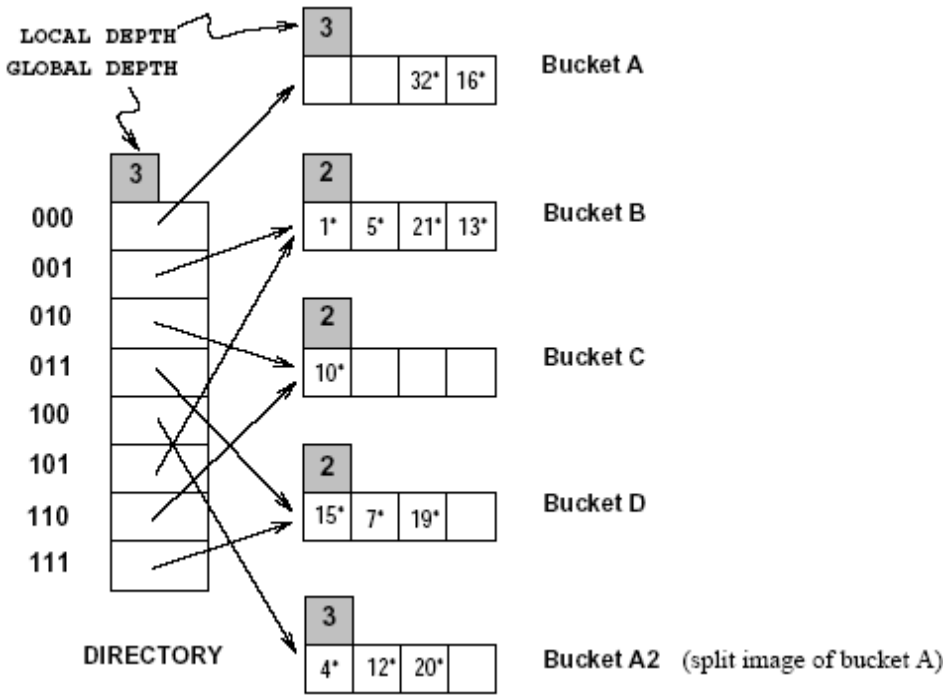


- Insert an entry with hash value 20.
    - Binary code 10100.

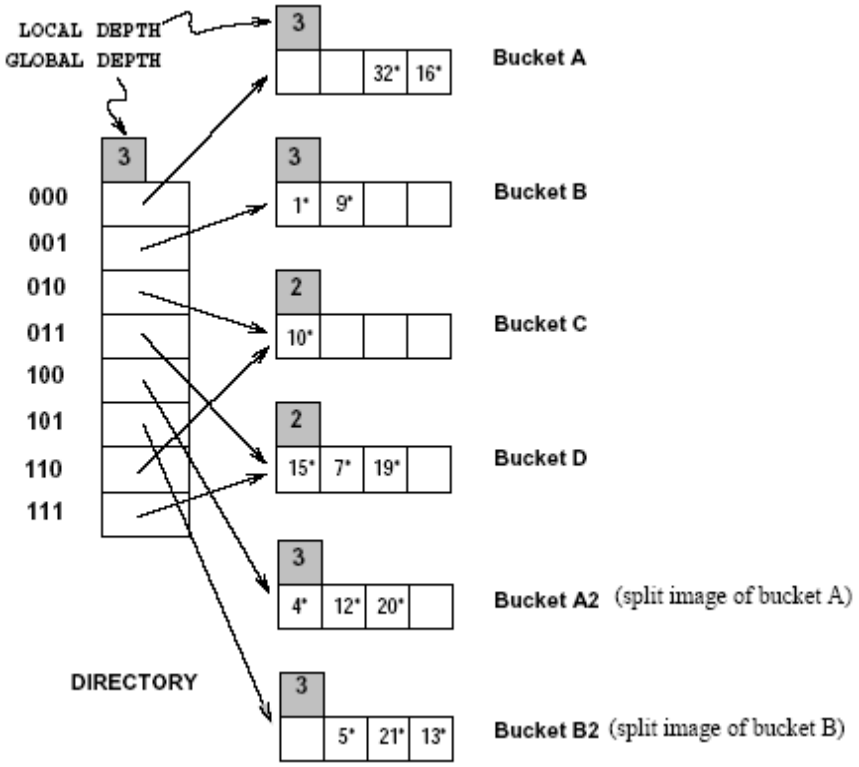
- Last two bits 00.
- Lead to bucket A.
- Bucket A is full.
- Split the bucket by allocating a new bucket, and redistribute the contents across the old bucket and its split image.



- We need three bits to discriminate between A and A2.
- The directory has only enough slots to store all two bit patterns.
- Double the directory.



- Whether splitting a bucket necessitates a directory doubling.
    - Not always
    - Ex:
        - Insert entry of hash value 9.
        - Belongs to bucket B.
        - Bucket is full split the bucket and use directory elements 001 and 101.
        - But if A and A2 are full and an insertion forces a bucket split, the directory is doubled.
    - To determine whether a directory doubling is needed, we maintain a local depth for each bucket.
    - If a bucket whose local depth is equal to the global depth is split, the directory must be doubled.
    - Initially all local depths are equal to the global depth.
    - Increase the global depth by 1, each time the directory is doubled.
    - Increase by 1 the local depth of the split bucket and assign this same local depth to its split image.



Linear Hashing

- LH handles the problem of long overflow chains without using a directory, and handles duplicates.
- Deals naturally with collision, and offers a lot of flexibility respect to the timing of bucket split.
- Use a family of hash functions **h**0, **h**1, **h**2, ... with the property that each function's range is twice that of its predecessor.

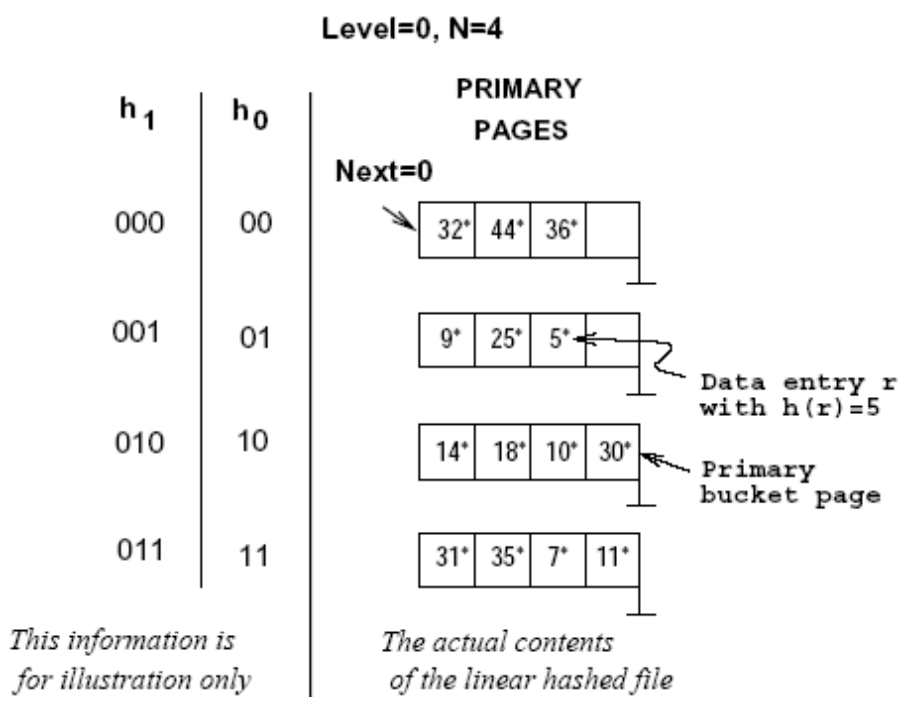
- If h_i maps a data entry into one of M buckets, h_{i+1} maps an entry into one of 2M buckets.
- h_i (value) = h(value) mod (2_i*N), N is the initial number of buckets, and h is the hash function.
- If N is chosen to be the power of 2, we apply h and look at the last d_i bits, d_0 is the number of bits needed to represent N and $d_i = d_0 + i$.
- Choose h to be a function that maps a data entry to some integer.
    - If N = 32, $d_0 = 5$, h_0 = h mod 32 ranging 0 to 31. $d_1 = d_0 + 1 = 6$ and h_1 = h mod (2*32)

Working out linear hashing

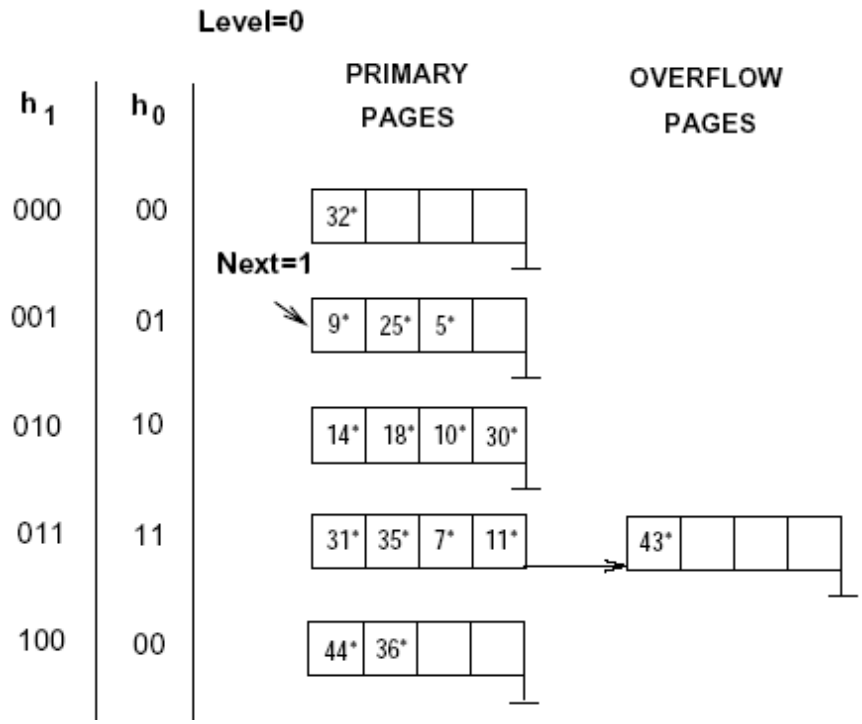
- A counter *level* is used to indicate the current round number and is initially 0.
- The bucket to split is denoted by *next* and is initially bucket 0.
- Denote the number of buckets in the file at the beginning of round *level* by NLevel

  NLevel = N * 2Level.
- Let the number of buckets at the beginning of round 0 denoted by N_0 be N.

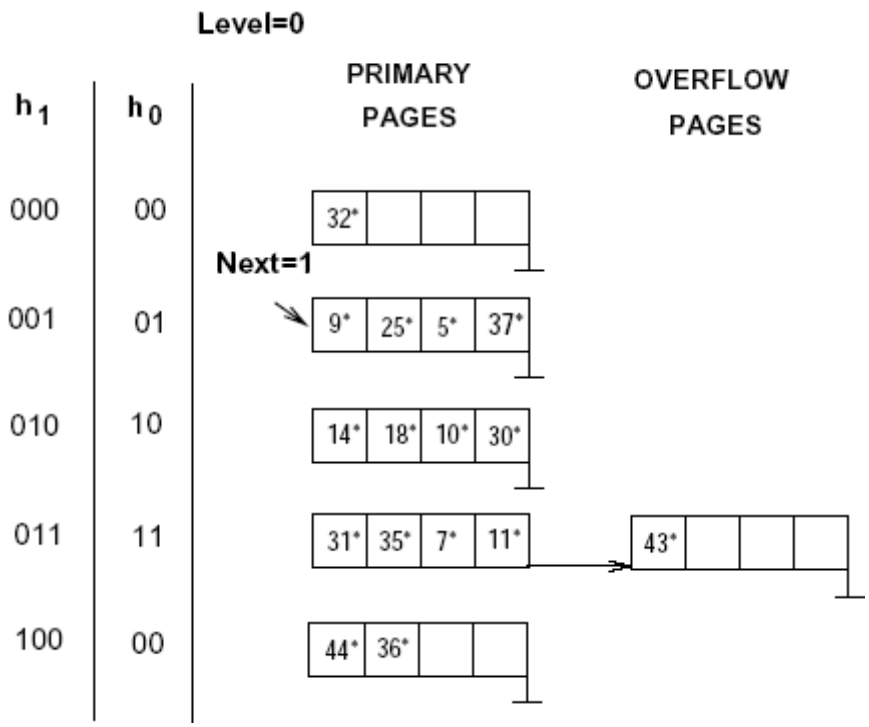
File contains four buckets: Each bucket can hold four data entries.

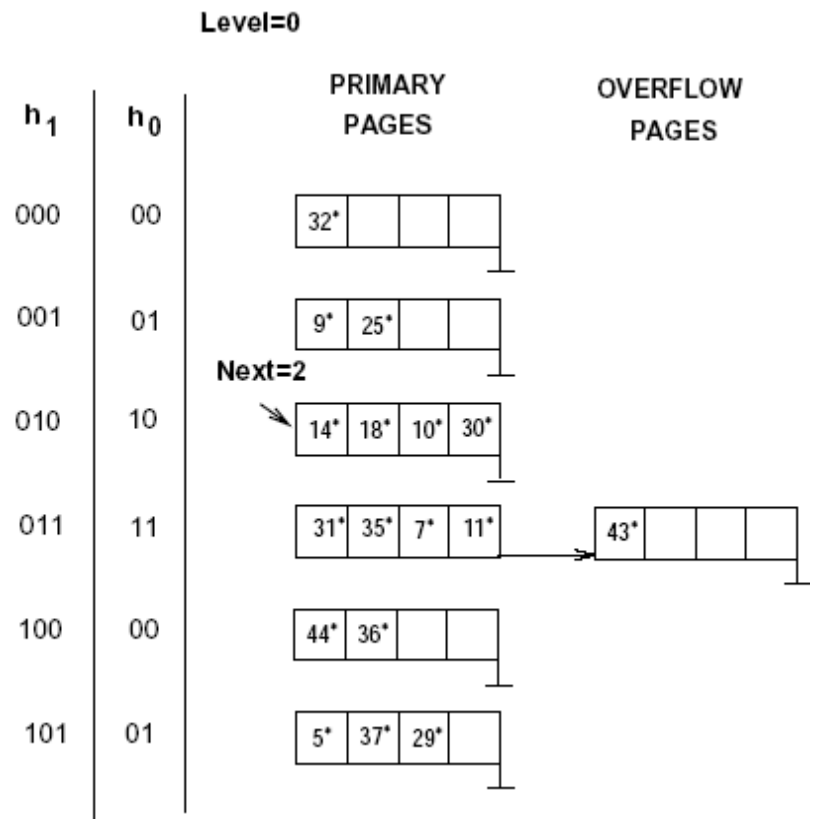


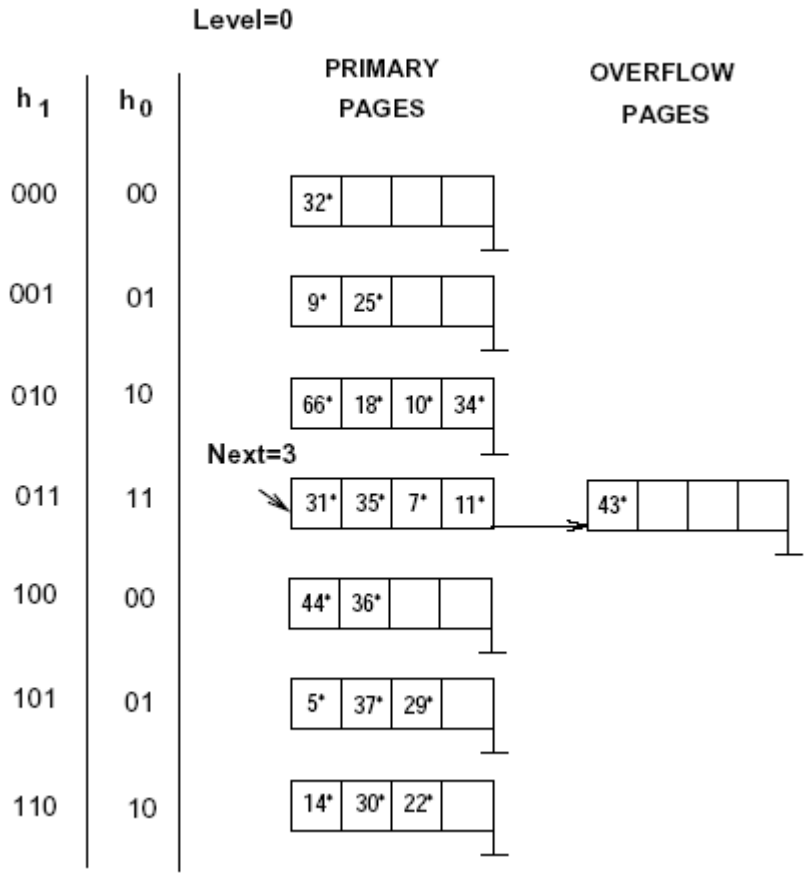
- A split occurs whenever a new overflow page is added.
- When a split is triggered the *next* bucket is split.
- The hash function hlevel +1 redistributes entries between this bucket and its split image.
- The *next* value is incremented by 1.
- Ex: Enter hash value 43.
    - Binary code 101011
    - Overflow occurs
    - Split bucket 1, redistribute entries.

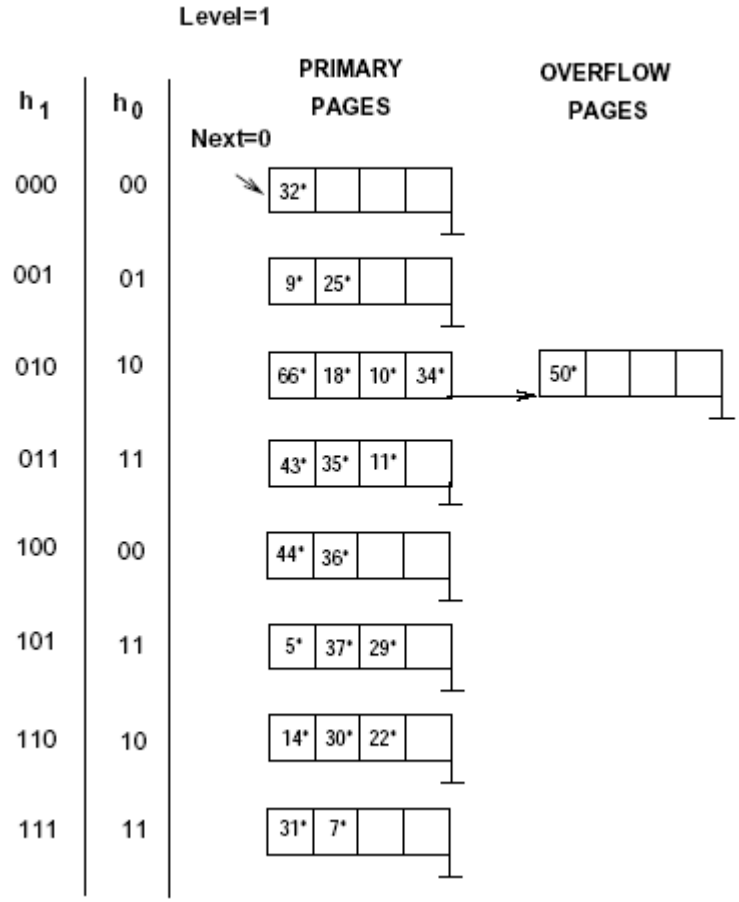


- At any time in the middle of a round *Level* all buckets above bucket *Next*, have been split.
- Buckets *next* through *Level* have not yet been split.
- If we use *hlevel* on a data entry and obtain a number b in the range *Next* through *NLevel* the data entry belongs to bucket b.
    - h_0(18), 10010, bucket is 2 since 2 is between the current value of *Next* (=1) and N1= (4)
    - h_0(32) = 100000
    - h_0(44) = 101100
    - bucket 0, since this value (0) is less than *Next* (=1), this bucket is split, so we have to apply h_1
    - h_1(32) = 100000
    - h_1(44) = 101100
    - 32 belongs to bucket 0 and 44 belongs to bucket 5.
- Not all insertions trigger a split.
    - Insert hash value 37
    - Binary code 100101
    - Insert in bucket 1, it has space.
- Now insert hash values 29, 22, 66 and 34
- Then insert 50.


Level=0
PRIMARY PAGES
OVERFLOW PAGES
h1
h0
000
00
32*
Next=1
001
01
9*
25*
5*
37*
010
10
14*
18*
10*
30*
011
11
31*
35*
7*
11*
43*
100
00
44*
36*


Level=0
PRIMARY PAGES
OVERFLOW PAGES
h1
h0
000
00
32*
001
01
9*
25*
Next=2
010
10
14*
18*
10*
30*
011
11
31*
35*
7*
11*
43*
100
00
44*
36*
101
01
5*
37*
29*


Level=0
PRIMARY PAGES
OVERFLOW PAGES
h 1
h 0
000
00
32*
001
01
9*
25*
010
10
66*
18*
10*
34*
Next=3
011
11
31*
35*
7*
11*
43*
100
00
44*
36*
101
01
5*
37*
29*
110
10
14*
30*
22*


Level=1
PRIMARY PAGES
OVERFLOW PAGES
h 1
h 0
Next=0
000
00
32*
001
01
9*
25*
010
10
66*
18*
10*
34*
50*
011
11
43*
35*
11*
100
00
44*
36*
101
11
5*
37*
29*
110
10
14*
30*
22*
111
11
31*
7*